Problem 1. Basic Concepts

For this problem, the goal is to explore basic concepts using the Neural Network playground: https://playground.tensorflow.org/.

1(a).

1.a.i. What do the orange and blue colors actually represent for the data input, the features, the hidden layers, and the output?

Solution

- **Data input:** Orange = class 1, Blue = class 0.
- **Features:** Orange = feature > 0, Blue = feature < 0, White = ≈ 0.
- **Hidden layers:** Orange = positive activation, Blue = negative activation, White = activation ≈ 0.
- **Output:** Orange = predicted class 1, Blue = predicted class 0.

1.a.ii. Change the activation function to ReLU. What does the white color represent for the data input, the features, the hidden layers, and the output?

Solution

- **White** indicates regions where the neuron output is exactly zero: $\text{ReLU}(z) = 0$.

1.a.iii. Write down the equation of the Neural Network represented by the default values.

Solution Let $\mathbf{x} = [x,\, y]^T$. The network has two hidden layers with tanh activation.

$$h^{(1)} = \tanh\Big(W^{(1)}\mathbf{x} + b^{(1)}\Big)$$

$$h^{(2)} = \tanh\Big(W^{(2)}h^{(1)} + b^{(2)}\Big)$$

$$\hat{y} = \sigma\Big(W^{(3)}h^{(2)} + b^{(3)}\Big)$$

$$\text{where } \sigma(z) = \frac{1}{1+e^{-z}}.$$

1(b). Train the Neural Network and answer the following:

1.b.i. What does it mean to train through an epoch?

Solution Training through one epoch means using every training sample exactly once to update the network parameters.

1.b.ii. How do you know that the network converged in terms of the test loss? Provide screenshots.

Solution Yes. The network has converged because the test loss stabilizes at a small value (`0.003`) and no longer decreases over additional epochs. The decision boundary is also stable. The screenshot in Fig. 1 shows the final test loss and prediction region.

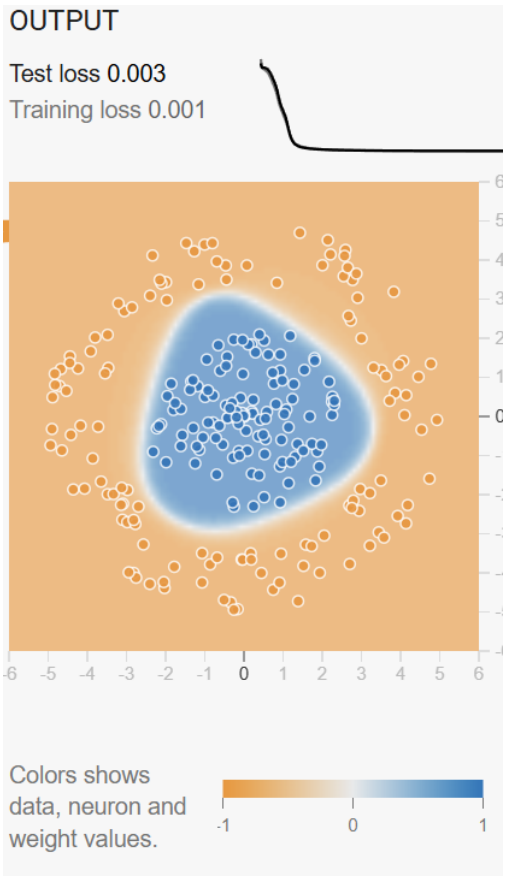

Figure 1: Test loss and decision boundary at convergence.

1.b.iii. What is the minimum number of epochs that it takes to converge? Explain your answer and provide screenshots.

Solution The minimum number of epochs required for convergence is the point after which the test loss no longer decreases. As shown in Fig. 2, the test loss stabilizes around epoch ≈ 170, indicating convergence.

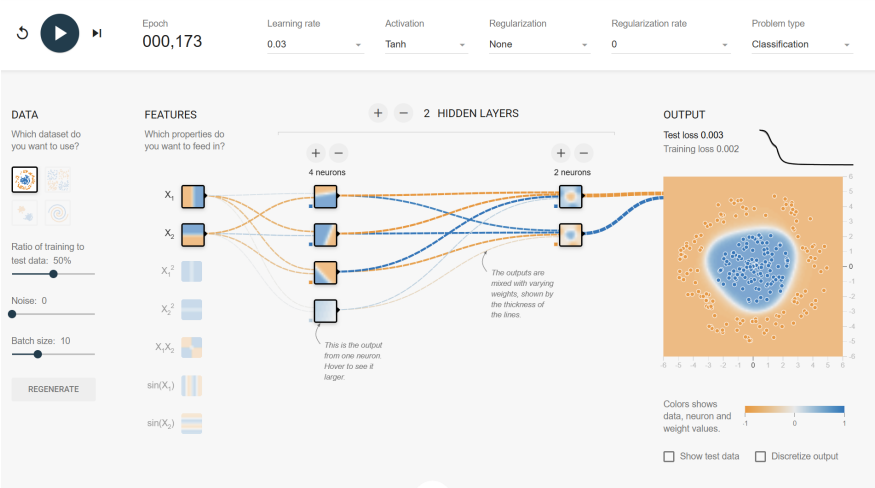

Figure 2: Test loss curve and decision boundary near convergence (epoch ≈ 173).

1.b.iv. Increase the learning rate by a factor of 100. What happened to the convergence? Explain. Provide screenshots and discuss convergence in terms of the number of epochs.

Solution Increasing the learning rate by a factor of 100 makes training unstable. As shown in Fig. 3, the test loss oscillates instead of decreasing smoothly, and the decision boundary becomes irregular. The model does not converge within a reasonable number of epochs.

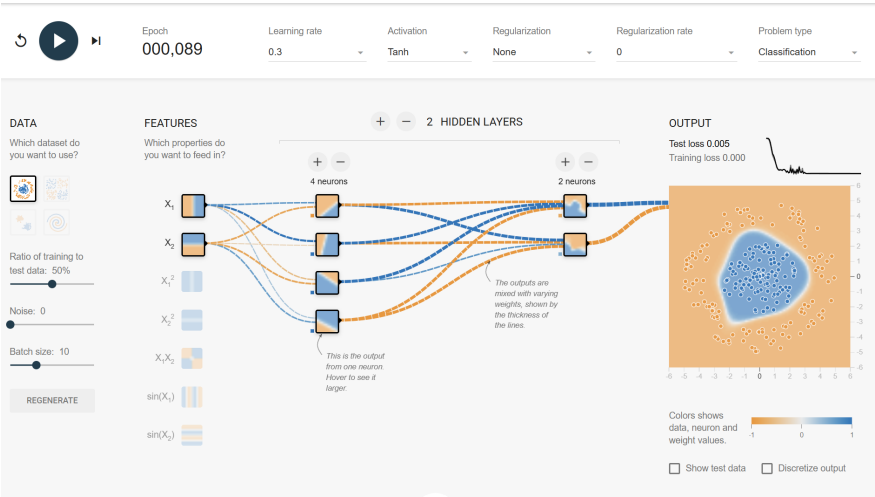

Figure 3: Training with learning rate increased by a factor of 100. The test loss oscillates and does not converge.

1.b.v. Repeat the previous question by reducing the learning rate to the lowest value.

Solution With the learning rate reduced to the lowest value, convergence becomes extremely slow. As shown in Fig. 4, both the test loss and decision boundary improve only gradually, requiring many more epochs. The model eventually moves in the correct direction, but at a very slow rate.

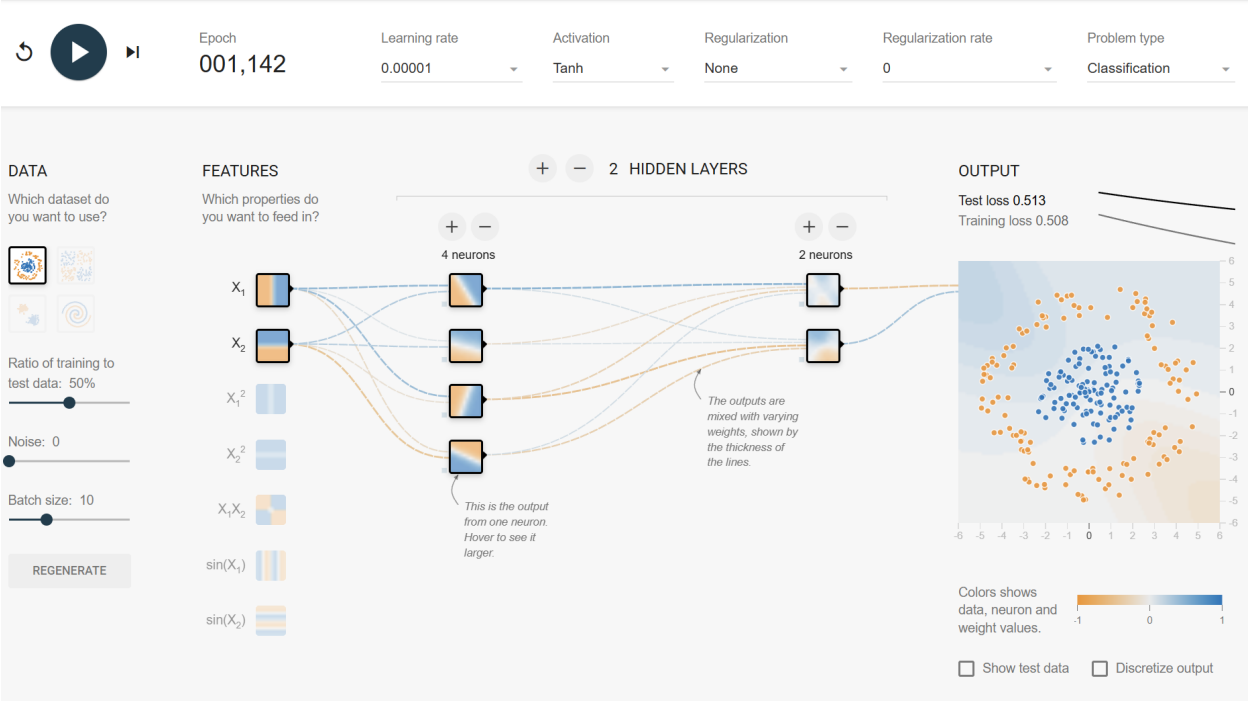

Figure 4: Training with the lowest learning rate. Convergence is very slow and the test loss remains high for many epochs.

Problem 2. Backpropagation

2(a). For this problem, we will follow the tutorial on autograd given at: PyTorch autograd.

We use a loss function to train neural networks using a $l(.)$ that takes a vector and outputs a scalar as in traditional optimization methods of studying $f(x)$. However, neural networks use function composition to propagate the inputs tothe outputs. In the autograd example, the notation is as follows:

- $l(\cdot)$: is the output scalar loss function (e.g., mean squared error).
- y: denotes the top output layers of the neural network.
- x: denotes the input to the neural network.

In order to train Neural-networks using the loss function, we need to compute gradients of $l(.)$ with respect to all of the variables. In the simplest form, we use functional composition to express $l(.)$ for a specific input: $l(y(x))$. When designing neural-networks, we can assume that the derivatives of each level are available to us. This means that we assume that the following derivatives are given to us:

$$\frac{\partial l}{\partial y_i}, \qquad \frac{\partial y_i}{\partial x_j} \quad \text{for } i = 1, \ldots, m \text{ and } j = 1, \ldots, n.$$

We then have to determine all other derivatives. To determine the dependence of the loss-function based on the input, we use:

$$\frac{\partial l}{\partial x} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_n} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix} \begin{bmatrix} \frac{\partial l}{\partial y_1} \\ \vdots \\ \frac{\partial l}{\partial y_m} \end{bmatrix}$$

Multiply out the matrix by the vector to derive an expression for $\frac{\partial l}{\partial x_i}$. How many terms do you have?

Solution The i-th component of $\dfrac{\partial l}{\partial x}$ is obtained by taking the dot product of the i-th row of the Jacobian $\dfrac{\partial y}{\partial x}$ with the vector $\dfrac{\partial l}{\partial y}$:

$$\frac{\partial l}{\partial x_i} = \sum_{k=1}^{m} \frac{\partial y_k}{\partial x_i} \frac{\partial l}{\partial y_k}, \qquad i = 1, \ldots, n.$$

There are m total terms in the expression for $\frac{\partial l}{\partial x_i}$, one for each output component y_k.

2(b). Let us introduce one more layer. Suppose that the loss is given by $l(y(g(x)))$. Apply 2(a) recursively to derive an expression for $\partial l/\partial x_i$ for this case. Assume that g outputs a p-dimensional vector that serves as the input to y. Assume that all other dimensions remain as given in 1(a). You will need to clearly label all matrix dimensions in your derivation.

Solution Let $x \in \mathbb{R}^n$, $g(x) \in \mathbb{R}^p$, and $y(g) \in \mathbb{R}^m$. Then

$$\frac{\partial l}{\partial y} \in \mathbb{R}^{1 \times m}, \quad \frac{\partial y}{\partial g} \in \mathbb{R}^{m \times p}, \quad \frac{\partial g}{\partial x} \in \mathbb{R}^{p \times n}.$$

By the chain rule,

$$\frac{\partial l}{\partial g} = \frac{\partial l}{\partial y}\frac{\partial y}{\partial g} \in \mathbb{R}^{1\times p}, \qquad \frac{\partial l}{\partial x} = \frac{\partial l}{\partial g}\frac{\partial g}{\partial x} \in \mathbb{R}^{1\times n}.$$

Component-wise,

$$\frac{\partial l}{\partial x_i} = \sum_{j=1}^{p}\sum_{k=1}^{m} \frac{\partial l}{\partial y_k}\frac{\partial y_k}{\partial g_j}\frac{\partial g_j}{\partial x_i}.$$

2(c). Repeat 1(b) for $l\big(y(g_2(g_1(x)))\big)$. Assume that the output of g_2 is p_2-dimensional and the output of g_1 is p_1-dimensional. Make sure to clearly indicate all matrix dimensions. **Solution** Let $x \in \mathbb{R}^n$, $g_1(x) \in \mathbb{R}^{p_1}$, $g_2(g_1) \in \mathbb{R}^{p_2}$, and $y(g_2) \in \mathbb{R}^m$, with scalar loss $l(y)$.
Then the relevant Jacobians are:

$$\frac{\partial l}{\partial y} \in \mathbb{R}^{1\times m}, \quad \frac{\partial y}{\partial g_2} \in \mathbb{R}^{m\times p_2}, \quad \frac{\partial g_2}{\partial g_1} \in \mathbb{R}^{p_2\times p_1}, \quad \frac{\partial g_1}{\partial x} \in \mathbb{R}^{p_1\times n}.$$

By the chain rule,

$$\frac{\partial l}{\partial x} = \frac{\partial l}{\partial y}\frac{\partial y}{\partial g_2}\frac{\partial g_2}{\partial g_1}\frac{\partial g_1}{\partial x} \in \mathbb{R}^{1\times n}.$$

Component-wise, for $i = 1, \ldots, n$,

$$\frac{\partial l}{\partial x_i} = \sum_{k=1}^{m}\sum_{j=1}^{p_2}\sum_{r=1}^{p_1} \frac{\partial l}{\partial y_k}\frac{\partial y_k}{\partial (g_2)_j}\frac{\partial (g_2)_j}{\partial (g_1)_r}\frac{\partial (g_1)_r}{\partial x_i}.$$

Problem 3. Automatic Differentiation
The tutorial on autograd in `autograd_tutorial.ipynb` shows an example of how to verify that automatic differentiation works for a simple function. Modify the example to compute

$$\frac{\partial}{\partial x_i}\left[\frac{1}{6}\sum_{i=1}^{6}(x_i-2)^2+x_i\right].$$

Verify that your example works by computing the tensor output for $\partial/\partial x_i$ when $x_i = 1$.
Solution. We consider the function

$$f(x) = \frac{1}{6}\sum_{i=1}^{6}(x_i-2)^2+x_i.$$

The derivative of the ith term is

$$\frac{\partial f}{\partial x_i} = \frac{1}{3}(x_i-2)+1.$$

Evaluating at $x_i = 1$ gives

$$\left.\frac{\partial f}{\partial x_i}\right|_{x_i=1} = \frac{2}{3}.$$

To verify this using `torch.autograd`, we define the tensor `x = torch.ones(6, requires_grad=True)` and compute:

`f = (1/6) * torch.sum((x - 2)**2) + torch.sum(x),` `f.backward()`.

The resulting gradient is

$$\nabla f(x) = \begin{bmatrix}0.6667 & 0.6667 & 0.6667 & 0.6667 & 0.6667 & 0.6667\end{bmatrix},$$

which matches the analytic value $\frac{2}{3}$ for each component.

Problem 4. The Basic Neural Network Interface
For this problem, please study `fahrenheit_to_celsius_corrected.ipynb` and `neural_networks_tutorial.ipynb`. The SGD implementation is given in: https://pytorch.org/docs/stable/optim.html

4(a). Modify the example and show how to compute the conversion from Fahrenheit to Celsius. This is the inverse of the conversion given in the example.
Solution. To learn the inverse temperature conversion, we fit the linear model

$$C = \alpha_0 + \alpha_1 F$$

using autograd and gradient descent. Random Fahrenheit samples are generated, their true Celsius values are computed from

$$C_{\text{true}} = \frac{F - 32}{1.8},$$

and the parameters α_0, α_1 are optimized by minimizing the mean–squared error. The training converges to the exact inverse mapping.

```
Fahrenheit to Celsius
True:    C = -17.78 + 0.56·F
Learned: C = -17.78 + 0.56·F
Final loss: 4.411049303598702e-11
Iterations: 2000
```

4(b). Translate the example into the neural-network framework and learn the parameters using:

- Stochastic Gradient Descent (SGD) without momentum,
- Stochastic Gradient Descent (SGD) with momentum.

Based on the documentation in https://pytorch.org/docs/stable/optim.html, the basic SGD update is

$$p_{t+1} = p_t - \text{lr} \cdot \nabla_p f,$$

where lr is the learning rate and p is a parameter being optimized.
With momentum, for step $t+1$, the approach is modified to:

$$v_{t+1} = \mu \cdot v_t + g_{t+1}, \qquad p_{t+1} = p_t - \text{lr} \cdot v_{t+1},$$

where $g_{t+1} = \nabla_p f_{t+1}$. You do not need to implement SGD manually—simply call the appropriate optimizer functions in PyTorch. Document your results for several different parameter choices.
Solution. We translate the Fahrenheit-to-Celsius conversion into the neural-network framework by using a single fully connected layer `nn.Linear(1,1)`, which implements the model

$$C = \alpha_0 + \alpha_1 F.$$

Using the same dataset as in part 4(a), we train this network twice with PyTorch's `optim.SGD`: first with standard SGD (no momentum), then with SGD including momentum ($\mu = 0.9$). In both cases we minimize the mean–squared error between predicted and true Celsius values.

For both optimizers, the learned parameters converge to

$$C \approx -17.78 + 0.56\,F,$$

matching the true inverse conversion. The final printed results are:

```
Problem 4(b): Neural network with SGD
SGD (no momentum)
Learned: C = -17.78 + 0.56·F
Final loss: 4.5929482439532876e-11

SGD (with momentum = 0.9)
Learned: C = -17.78 + 0.56·F
Final loss: 8.45830089996058e-12
```

Problem 5. Adam Optimization

This problem is based on *ADAM: A Method for Stochastic Optimization*, which can be accessed at https://arxiv.org/pdf/1412.6980. Additionally, consult the `ML-intro.pdf` document.

5(a). Consider the update for the mean that is given by the Adam algorithm. Use the definition of the sample mean to derive an expression for β_1 that makes the bias update correct.

Solution

Adam updates the first moment using

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t.$$

The unbiased sample mean of the gradients is

$$\bar{g}_t = \frac{1}{t}\sum_{i=1}^{t} g_i.$$

Expanding the Adam update gives

$$m_t = (1-\beta_1)\sum_{i=1}^{t} \beta_1^{t-i} g_i.$$

To match the uniform weight $1/t$ in the sample mean, we require

$$1-\beta_1 = \frac{1}{t},$$

so the correct choice is

$$\beta_1 = 1 - \frac{1}{t}.$$

5(b). Does your expression for β_1 in 5(a) depend on the batch size n? What happens when you double the batch size?

Solution

The expression derived in part 5(a),

$$\beta_1 = 1 - \frac{1}{t},$$

depends only on the iteration index t and does not involve the batch size n. Doubling the batch size changes the variance of each gradient estimate but does not change the form of the bias in the exponential moving average. Thus, the value of β_1 required to correct the bias remains the same regardless of batch size.

5(c). Consider the update for the sample variance given by the Adam algorithm.

5.c.i. Use the definition of the sample variance to derive an expression for β_2.

Solution

Adam updates the second moment via

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2.$$

The unbiased sample variance estimate of the squared gradients is

$$\frac{1}{t}\sum_{i=1}^{t} g_i^2.$$

Expanding the Adam update gives

$$v_t = (1 - \beta_2) \sum_{i=1}^{t} \beta_2^{t-i} g_i^2.$$

To match the uniform weight $1/t$ used in the sample variance, we require

$$1 - \beta_2 = \frac{1}{t},$$

which yields

$$\beta_2 = 1 - \frac{1}{t}.$$

Thus, the same bias condition applies to the second-moment estimate.

5.c.ii. What did you assume for the sample mean in order to derive your expression?
Solution
The derivation assumed that the sample mean used in the variance formula was the unbiased average

$$\bar{g}_t = \frac{1}{t} \sum_{i=1}^{t} g_i,$$

so that both the mean and variance rely on uniform weights $1/t$.

5.c.iii. Why do we need a bias correction term for the second moment?
Solution
Without bias correction, the exponential moving average $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ is biased low during early iterations because it begins at zero and places most weight on that initial value. The bias correction term removes this initialization bias so that v_t becomes an unbiased estimate of the true second moment.

5.c.iv. Is the assumption in 5.c.i correct at the initial epochs?
Solution
No. In the early epochs only a few gradients are available, so the sample mean is not yet accurate. This makes the assumption in 5.c.i invalid at the start.

5.c.v. How about when you approach convergence?
Solution
Near convergence, many gradients have been observed, so the sample mean becomes accurate and the assumption in 5.c.i holds much more closely.

5(d). What happens to the variance of the sample mean if we double the batch size?
Solution
Doubling the batch size reduces the variance of the sample mean by a factor of two, since averaging over twice as many samples cuts the variance in half.

5(e). Give the general shape of the n-dimensional covariance matrix that the Adam algorithm assumes. When is this assumption violated?

Solution

Adam assumes that the covariance matrix of the gradients is diagonal, i.e.,

$$\Sigma = \begin{bmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^2 \end{bmatrix}.$$

This assumes all gradient components are uncorrelated. The assumption is violated when parameters exhibit significant cross-correlations, causing nonzero off-diagonal terms in the true covariance matrix.

5(f). Suppose that the variance of the gradient for a neural network weight component is very small. Derive a simplified expression for the final expression of parameter update.

Solution

In Adam, the parameter update is

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

If the gradient variance is very small, then $v_t \approx g_t^2$ and $\sqrt{\hat{v}_t} \approx |g_t|$. Thus the update becomes

$$\theta_{t+1} \approx \theta_t - \alpha \frac{\hat{m}_t}{|g_t| + \varepsilon}.$$

Since $\hat{m}_t \approx g_t$ when variance is small, the expression simplifies to

$$\theta_{t+1} \approx \theta_t - \alpha \operatorname{sign}(g_t).$$

Thus, with very small gradient variance, Adam behaves like a sign-based optimizer.

5(h). Under the assumptions made here, show that the step size is scaling the gradient update to ensure it has unit variance.

Solution

With small gradient variance, $\hat{m}_t \approx g_t$ and $\hat{v}_t \approx g_t^2$. Then

$$\frac{\hat{m}_t}{\sqrt{\hat{v}_t}} \approx \frac{g_t}{|g_t|} = \operatorname{sign}(g_t),$$

which has unit variance. Thus the step size rescales the gradient so that the update has variance one.

5(e) Consider the optimal step-size derived for steepest-descent:

$$f(x) = \frac{1}{2}x^T Q x - b^T x,$$

$$\alpha_k = \frac{\nabla f_k^T \nabla f_k}{\nabla f_k^T Q \nabla f_k}, \qquad x_{k+1} = x_k - \alpha_k \nabla f_k.$$

Step schedulers reduce α_k when we approach convergence. Consider the simple case where we reduce the scheduler by a $c\alpha_k$ in epochs e_1, e_2, e_3. Assume models Q_1, Q_2, Q_3 for each one of the epochs. Derive expressions for Q_2, Q_3 based on Q_1. Sketch a 2-D example of what the models are assuming to be an ideal case for $c = 0.5$.

Solution

Reducing the step size by a factor c gives the updated rule

$$c\alpha_k = \frac{\nabla f_k^T \nabla f_k}{\nabla f_k^T Q_2 \nabla f_k}.$$

Matching this with the original formula implies

$$Q_2 = \frac{1}{c} Q_1, \qquad Q_3 = \frac{1}{c^2} Q_1.$$

For $c = 0.5$, this gives $Q_2 = 2Q_1$ and $Q_3 = 4Q_1$. The models therefore assume increasing curvature as the scheduler reduces the step size.

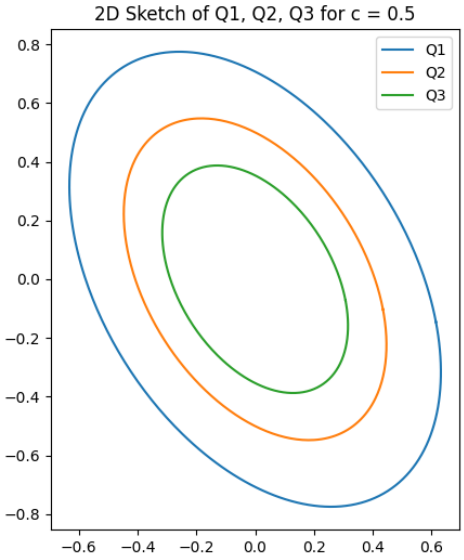

Figure 5: 2-D sketch of $x^T Q_1 x = 1$, $x^T Q_2 x = 1$, and $x^T Q_3 x = 1$ for $c = 0.5$.

```
import torch
import torch.optim as optim
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np

def get_loss(c_pred, c_true):
return torch.mean((c_pred - c_true)**2)

def predict(alpha0, alpha1, f):
return alpha0 + alpha1 * f

# ============================================================
# Problem 3  Automatic Differentiation
# ============================================================

x = torch.ones(6, requires_grad=True)
f = (1/6) * torch.sum((x - 2)**2) + torch.sum(x)
f.backward()
print("Problem␣3␣gradient:", x.grad)

# ============================================================
# Problem 4(a)  Fahrenheit  Celsius using Autograd
# ============================================================

f_vals = 100 * torch.rand(10) + 32
c_true = (f_vals - 32) / 1.8
f_mean = f_vals.mean()
f_std = f_vals.std()
f_norm = (f_vals - f_mean) / f_std

beta0 = torch.zeros(1, requires_grad=True)
beta1 = torch.zeros(1, requires_grad=True)
optimizer = optim.SGD([beta0, beta1], lr=0.1)

max_iters = 2000
loss = None

for iteration in range(max_iters):
optimizer.zero_grad()
c_pred_norm = beta0 + beta1 * f_norm
loss = get_loss(c_pred_norm, c_true)
loss.backward()
optimizer.step()

with torch.no_grad():
alpha1 = beta1 / f_std
alpha0 = beta0 - beta1 * f_mean / f_std

true_formula = "True:␣␣␣␣C␣=␣-17.78␣+␣0.56F"
learned_formula = f"Learned:␣C␣=␣{alpha0.item():.2f}␣+␣{alpha1.item():.2f}F
    "

print("\nFahrenheit␣to␣Celsius")
```

```
print("\t" + true_formula)
print("\t" + learned_formula)
print("Final␣loss:", loss.item())
print("Iterations:", max_iters)

# ============================================================
# Problem 4(b)  Neural-Net Framework with SGD and Momentum
# ============================================================

class LinearNet(nn.Module):
def __init__(self):
super(LinearNet, self).__init__()
self.linear = nn.Linear(1, 1)

def forward(self, x):
return self.linear(x)

f_input_norm = f_norm.unsqueeze(1)
c_target = c_true.unsqueeze(1)
criterion = nn.MSELoss()

net_sgd = LinearNet()
optimizer_sgd = optim.SGD(net_sgd.parameters(), lr=0.1, momentum=0.0)
num_epochs = 2000

for epoch in range(num_epochs):
optimizer_sgd.zero_grad()
c_pred = net_sgd(f_input_norm)
loss_sgd = criterion(c_pred, c_target)
loss_sgd.backward()
optimizer_sgd.step()

with torch.no_grad():
w_sgd = net_sgd.linear.weight.item()
b_sgd = net_sgd.linear.bias.item()
alpha1_sgd = w_sgd / f_std
alpha0_sgd = b_sgd - w_sgd * f_mean / f_std

net_mom = LinearNet()
optimizer_mom = optim.SGD(net_mom.parameters(), lr=0.1, momentum=0.9)

for epoch in range(num_epochs):
optimizer_mom.zero_grad()
c_pred_m = net_mom(f_input_norm)
loss_mom = criterion(c_pred_m, c_target)
loss_mom.backward()
optimizer_mom.step()

with torch.no_grad():
w_mom = net_mom.linear.weight.item()
b_mom = net_mom.linear.bias.item()
alpha1_mom = w_mom / f_std
alpha0_mom = b_mom - w_mom * f_mean / f_std
```

```
print("\nProblem 4(b): Neural network with SGD")
print("SGD (no momentum)")
print(f"\tLearned: C = {alpha0_sgd.item():.2f} + {alpha1_sgd.item():.2f}F")
print(f"\tFinal loss: {loss_sgd.item()}")

print("\nSGD (with momentum = 0.9)")
print(f"\tLearned: C = {alpha0_mom.item():.2f} + {alpha1_mom.item():.2f}F")
print(f"\tFinal loss: {loss_mom.item()}")

# ============================================================
# Problem 5(e)  Q1, Q2, Q3 Sketch
# ============================================================

c = 0.5
Q1 = np.array([[3, 1],[1, 2]])
Q2 = Q1 / c
Q3 = Q1 / (c**2)

def plot_ellipse(Q, ax, label):
theta = np.linspace(0, 2*np.pi, 400)
unit_circle = np.vstack([np.cos(theta), np.sin(theta)])
eigvals, eigvecs = np.linalg.eigh(Q)
D_inv_sqrt = np.diag(1/np.sqrt(eigvals))
transform = eigvecs @ D_inv_sqrt @ eigvecs.T
ellipse = transform @ unit_circle
ax.plot(ellipse[0], ellipse[1], label=label)

fig, ax = plt.subplots(figsize=(6,6))
plot_ellipse(Q1, ax, "Q1")
plot_ellipse(Q2, ax, "Q2")
plot_ellipse(Q3, ax, "Q3")
ax.set_aspect('equal')
ax.set_title("2D Sketch of Q1, Q2, Q3 for c = 0.5")
ax.legend()
plt.savefig("plots/fig_5.png")
plt.show()
```